

Pontifícia Universidade Católica de Minas Gerais

ICEI – Instituto de Ciências Exatas e Informática

Redes de Computadores I

Desenvolvimento de uma Aplicação em Rede Utilizando Java

Java Network Monitor

Felipe Barros Ratton de Almeida

Laura Menezes Heráclito Alves

Mateus Ribeiro Fernandes

Matheus Henrique Gonçalves

Vitor de Meira Gomes

Professor: Max do Val Machado

5º Período – 2026

Repositório: https://github.com/LINK_DO_GRUPO/java-network-monitor

Belo Horizonte, 2026

Resumo

Este trabalho apresenta o desenvolvimento de uma aplicação distribuída de monitoramento de máquinas em rede, implementada em Java, com interface gráfica Java Swing, comunicação via protocolos TCP e UDP, e processamento concorrente com multithreading. A aplicação segue a arquitetura cliente-servidor: o cliente monitora métricas da máquina local (CPU, memória e disco) e as envia ao servidor, que exibe as informações em tempo real. A validação foi realizada em uma topologia de três redes distintas, configurada no Cisco Packet Tracer com três roteadores 2911 e roteamento estático, e o tráfego foi analisado com o Wireshark. Os resultados confirmam a correta comunicação entre redes distintas e o funcionamento adequado de ambos os protocolos de transporte.

Palavras-chave: Java; TCP; UDP; Multithreading; Java Swing; Redes de Computadores; Cisco Packet Tracer; Wireshark.

Sumário

1	Introdução	3
2	Objetivos	3
3	Fundamentação Teórica	3
3.1	Protocolo TCP	3
3.2	Protocolo UDP	3
3.3	Multithreading	4
3.4	Roteamento Estático	4
4	Implementação	4
4.1	Estrutura do Projeto	4
4.2	Servidor TCP	4
4.3	Servidor UDP	5
4.4	Protocolo de Comunicação	6
5	Configuração da Rede – Cisco Packet Tracer	6
5.1	Topologia	6
5.2	Endereçamento IP	6
5.3	Configuração dos Roteadores	6
5.3.1	Router3 – Rede 192.168.0.0/16	6
5.3.2	Router4 – Rede de interligação	7
5.3.3	Router5 – Rede 10.0.0.0/8	7
5.4	Validação da Conectividade	7
6	Análise com Wireshark	8
6.1	Tráfego TCP – Porta 5000	8
6.2	Stream TCP – Protocolo de Mensagens	8
6.3	Tráfego UDP – Porta 5001	9
7	Principais Desafios	11
8	Conclusão	11
	Uso de Inteligência Artificial	11

1 Introdução

A comunicação em rede é um dos pilares fundamentais dos sistemas computacionais modernos. Aplicações distribuídas dependem de protocolos de transporte confiáveis e eficientes para garantir a troca de informações entre máquinas em ambientes heterogêneos.

Este trabalho tem como objetivo o desenvolvimento de uma aplicação de monitoramento de máquinas em rede utilizando a linguagem Java, explorando os protocolos TCP (*Transmission Control Protocol*) e UDP (*User Datagram Protocol*), implementação de multithreading e desenvolvimento de interface gráfica com Java Swing.

A aplicação, denominada *Java Network Monitor*, permite que máquinas clientes registrem-se em um servidor central e enviem periodicamente suas métricas de desempenho. O servidor exibe essas informações em tempo real em um painel gráfico.

O código-fonte completo está disponível em:

https://github.com/LINK_DO_GRUPO/java-network-monitor

2 Objetivos

- Desenvolver uma aplicação cliente-servidor em Java;
- Implementar comunicação TCP para registro e troca de mensagens confiáveis (porta 5000);
- Implementar comunicação UDP para envio periódico de métricas – heartbeat (porta 5001);
- Utilizar multithreading para atender múltiplos clientes simultaneamente;
- Desenvolver interface gráfica com Java Swing;
- Configurar topologia de três redes distintas no Cisco Packet Tracer com roteamento estático;
- Validar a comunicação entre redes e analisar o tráfego com o Wireshark.

3 Fundamentação Teórica

3.1 Protocolo TCP

O TCP é um protocolo orientado à conexão que garante entrega confiável, ordenada e com verificação de erros. Antes de qualquer troca de dados, é estabelecida uma conexão por meio do *three-way handshake* (SYN, SYN-ACK, ACK). No projeto, o TCP foi utilizado para o registro dos clientes e para a troca de mensagens estruturadas de monitoramento, onde a confiabilidade é essencial.

3.2 Protocolo UDP

O UDP é um protocolo sem conexão, sem garantia de entrega ou ordenamento, com baixa latência e menor overhead. No projeto, o UDP foi utilizado para o envio periódico de mensagens de *heartbeat* com métricas de status. A perda eventual de um pacote UDP

não compromete o funcionamento da aplicação, tornando-o adequado para esse caso de uso.

3.3 Multithreading

O uso de múltiplas threads permite que o servidor atenda vários clientes simultaneamente. Cada cliente TCP conectado é tratado por uma `ClientHandler` dedicada, e o servidor UDP opera em thread separada e paralela.

3.4 Roteamento Estático

O roteamento estático consiste na configuração manual de rotas nos roteadores, indicando por qual interface os pacotes destinados a cada rede devem ser encaminhados. Foram configurados três roteadores com roteamento estático para permitir comunicação entre as redes 192.168.0.0/16, 172.16.0.0/12 e 10.0.0.0/8.

4 Implementação

4.1 Estrutura do Projeto

```
src/
  client/
    ClientMain.java
    network/ (TcpClient.java, UdpClient.java)
    gui/      (ConnectionFrame.java, MainFrame.java, components/)
    model/    (MachineStatus.java)
  common/
    dto/      (RegisterMessage, StatusMessage, HeartbeatMessage)
    protocol/ (MessageType.java)
    utils/    (Constants.java)
  server/
    ServerMain.java
    network/  (TcpServer.java, UdpServer.java, ClientHandler.java)
    service/  (MonitoringService.java)
    model/    (ConnectedMachine.java)
```

4.2 Servidor TCP

O `TcpServer` estende `Thread` e aguarda conexões na porta 5000. Para cada cliente aceito, instancia uma nova `ClientHandler`:

Listing 1: `TcpServer` – aceite de conexões

```
1 while (true) {
2     Socket clientSocket = serverSocket.accept();
3     ClientHandler handler = new ClientHandler(clientSocket,
4         monitoringService);
5     handler.start(); // nova thread por cliente
}
```

O `ClientHandler` processa os comandos recebidos via TCP seguindo o protocolo textual definido:

Listing 2: `ClientHandler` – processamento de mensagens

```
1 switch (command) {  
2     case "REGISTER":  
3         monitoringService.registerMachine(machineName, clientIp);  
4         return "REGISTER_OK|" + machineName;  
5     case "GET_MACHINE_LIST":  
6         return monitoringService.getMachineList();  
7     case "DISCONNECT":  
8         return "DISCONNECT_OK";  
9 }
```

4.3 Servidor UDP

O `UdpServer` também estende `Thread` e aguarda datagramas na porta 5001. Processa dois tipos de mensagem UDP: `HEARTBEAT` e `STATUS`:

Listing 3: `UdpServer` – recepção de datagramas

```
1 DatagramSocket socket = new DatagramSocket(port);  
2 byte[] buffer = new byte[1024];  
3 while (true) {  
4     DatagramPacket packet = new DatagramPacket(buffer, buffer.  
5         length);  
6     socket.receive(packet);  
7     String message = new String(packet.getData(), 0, packet.  
8         getLength());  
9     processMessage(message);  
10 }
```

A mensagem `STATUS` carrega nome da máquina, CPU, memória e disco:

Listing 4: `UdpClient` – envio de status

```
1 public void sendStatus(String machineName, int cpu, int ram, int  
2     disk) {  
3     String message = "STATUS|" + machineName + "|" + cpu + "|" +  
4         ram + "|" + disk;  
5     sendPacket(message);  
6 }
```

4.4 Protocolo de Comunicação

Tabela 1: Tipos de mensagem do protocolo

Tipo	Protocolo	Descrição
REGISTER nome	TCP	Registro do cliente
REGISTER_OK nome	TCP	Confirmação de registro
GET_MACHINE_LIST	TCP	Solicita lista de máquinas
MACHINE_LIST nome,ip,cpu,mem,disk	TCP	Resposta com métricas
HEARTBEAT nome	UDP	Sinal de vida periódico
STATUS nome cpu mem disk	UDP	Métricas detalhadas
DISCONNECT	TCP	Encerramento de conexão

5 Configuração da Rede – Cisco Packet Tracer

5.1 Topologia

A topologia foi configurada com três roteadores (Router3, Router4 e Router5, modelo 2911) e dois computadores (PC0 e PC1), conectados por cabos de par trançado:

PC0 (192.168.1.10) -- R3 -- R4 -- R5 -- PC1 (10.0.0.10)

5.2 Endereçamento IP

Tabela 2: Endereçamento IP da topologia

Dispositivo	Interface	IP	Máscara
PC0	FastEthernet0	192.168.1.10	255.255.0.0
Router3	GigabitEthernet0/0	192.168.1.1	255.255.0.0
Router3	GigabitEthernet0/1	10.1.0.1	255.255.0.0
Router4	GigabitEthernet0/0	10.1.0.2	255.255.0.0
Router4	GigabitEthernet0/1	172.16.2.1	255.255.255.0
Router5	GigabitEthernet0/0	172.16.2.2	255.255.255.0
Router5	GigabitEthernet0/1	10.0.0.1	255.0.0.0
PC1	FastEthernet0	10.0.0.10	255.0.0.0

5.3 Configuração dos Roteadores

5.3.1 Router3 – Rede 192.168.0.0/16

Listing 5: Configuração do Router3

```

1 enable
2 configure terminal
3 interface GigabitEthernet0/0
4   ip address 192.168.1.1 255.255.0.0
5   no shutdown
6 exit

```

```
7 interface GigabitEthernet0/1
8   ip address 10.1.0.1 255.255.0.0
9   no shutdown
10 exit
11 ip route 172.16.2.0 255.255.255.0 10.1.0.2
12 ip route 10.0.0.0 255.0.0.0 10.1.0.2
13 end
```

5.3.2 Router4 – Rede de interligação

Listing 6: Configuração do Router4

```
1 enable
2 configure terminal
3 interface GigabitEthernet0/0
4   ip address 10.1.0.2 255.255.0.0
5   no shutdown
6 exit
7 interface GigabitEthernet0/1
8   ip address 172.16.2.1 255.255.255.0
9   no shutdown
10 exit
11 ip route 192.168.0.0 255.255.0.0 10.1.0.1
12 ip route 10.0.0.0 255.0.0.0 172.16.2.2
13 end
```

5.3.3 Router5 – Rede 10.0.0.0/8

Listing 7: Configuração do Router5

```
1 enable
2 configure terminal
3 interface GigabitEthernet0/0
4   ip address 172.16.2.2 255.255.255.0
5   no shutdown
6 exit
7 interface GigabitEthernet0/1
8   ip address 10.0.0.1 255.0.0.0
9   no shutdown
10 exit
11 ip route 192.168.0.0 255.255.0.0 172.16.2.1
12 ip route 172.16.0.0 255.240.0.0 172.16.2.1
13 end
```

5.4 Validação da Conectividade

Após a configuração, o teste de conectividade foi realizado com ping no PC0:

```
C:\>ping 10.0.0.10
Reply from 10.0.0.10: bytes=32 time<1ms TTL=125
Reply from 10.0.0.10: bytes=32 time<1ms TTL=125
```


A comunicação foi estabelecida entre as três redes distintas, validando o roteamento estático configurado.

6 Análise com Wireshark

6.1 Tráfego TCP – Porta 5000

A Figura 1 apresenta a captura do tráfego TCP na porta 5000, filtrado por `tcp.port == 5000 or udp.port == 5001`.

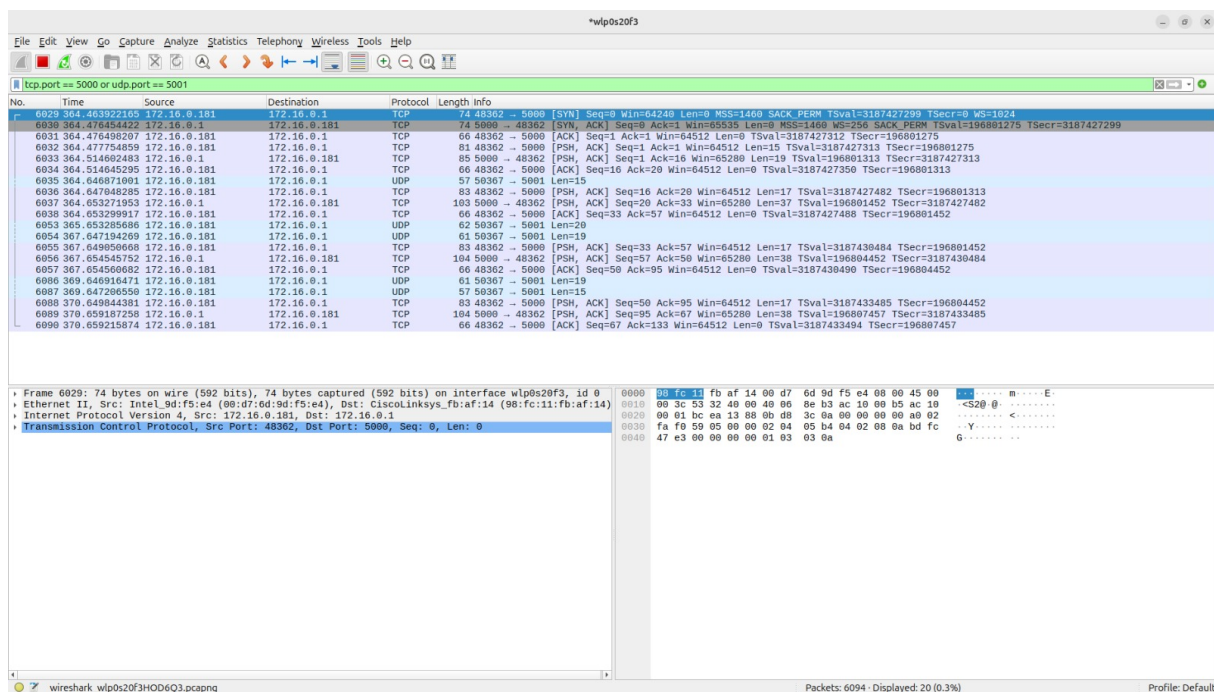


Figura 1: Captura Wireshark – tráfego TCP (porta 5000) e UDP (porta 5001). O pacote destacado em azul (6029) mostra o SYN do handshake TCP, com origem em 172.16.0.181 e destino 172.16.0.1 na porta 5000.

Na Figura 1, observa-se o *three-way handshake* TCP nos primeiros pacotes (SYN, SYN-ACK, ACK), confirmando o estabelecimento correto da conexão entre cliente (172.16.0.181) e servidor (172.16.0.1). Os pacotes subsequentes com flags [PSH, ACK] transportam as mensagens do protocolo de monitoramento. Intercalados, os pacotes UDP na porta 5001 confirmam o funcionamento simultâneo dos dois protocolos.

6.2 Stream TCP – Protocolo de Mensagens

A Figura 2 apresenta o conteúdo do stream TCP 23, obtido pela função *Follow TCP Stream* do Wireshark.

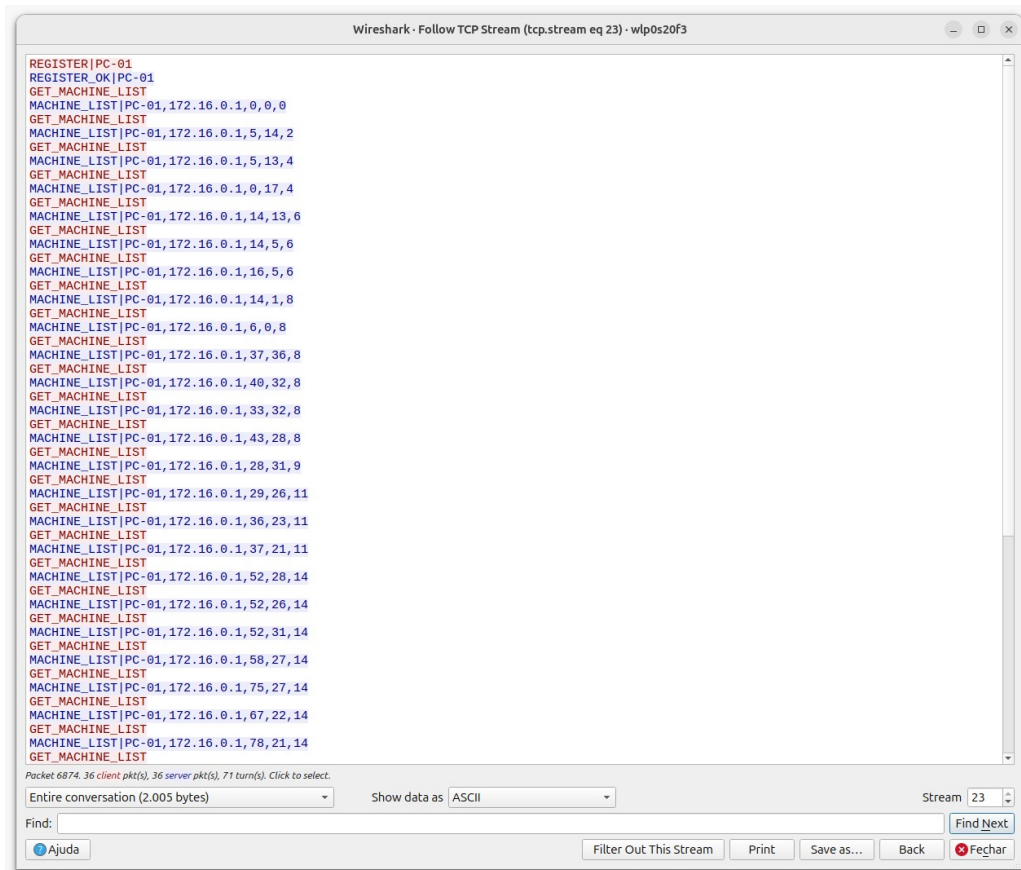


Figura 2: Conteúdo do stream TCP 23: troca completa de mensagens do protocolo de monitoramento entre cliente (vermelho) e servidor (azul).

O stream evidencia com clareza o protocolo em funcionamento. O cliente envia `REGISTER|PC-01` e o servidor responde com `REGISTER_OK|PC-01`, confirmando o registro. Em seguida, o cliente envia repetidamente `GET_MACHINE_LIST` e o servidor responde com `MACHINE_LIST|PC-01, 172.16.0.1,X,Y,Z`, onde os valores numéricos correspondem às métricas de CPU, memória e disco atualizadas a cada ciclo. O stream registra 36 pacotes do cliente e 36 do servidor (2.005 bytes no total), demonstrando a comunicação bidirecional contínua e correta via TCP.

6.3 Tráfego UDP – Porta 5001

A Figura ?? apresenta a captura filtrada por `udp.port == 5001`, isolando as mensagens de heartbeat/status enviadas pelo cliente ao servidor.

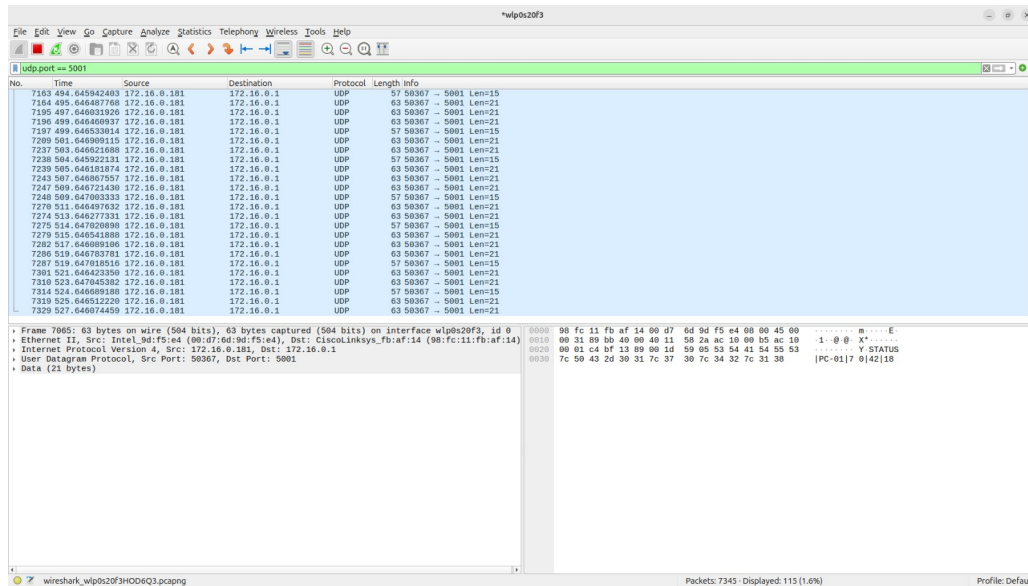


Figura 3: Captura Wireshark filtrada por `udp.port == 5001`. Datagramas periódicos de origem `172.16.0.181` para `172.16.0.1`, com tamanhos de 15 e 21 bytes alternados.

Os datagramas UDP são enviados periodicamente com dois tamanhos observados: 15 bytes (`HEARTBEAT|PC-01`) e 21 bytes (`STATUS|PC-01|X|Y|Z`). A regularidade dos envios – a cada aproximadamente 2 segundos – confirma o funcionamento do *scheduler* de métricas no cliente. A Figura 4 detalha o conteúdo de um pacote UDP individual.

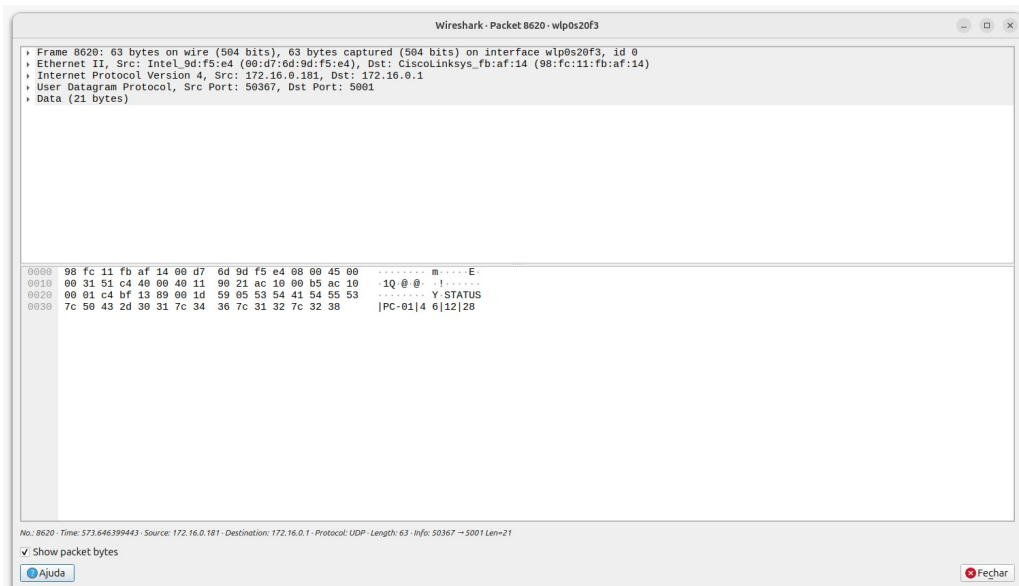


Figura 4: Detalhe do pacote UDP 8620: protocolo UDP com origem `172.16.0.181:50367` e destino `172.16.0.1:5001`, 21 bytes de dados (`STATUS|PC-01|...`).

O campo de dados decodificado em ASCII (`|PC-01|...STATUS...`) confirma que a mensagem `STATUS` foi transmitida corretamente e chegou ao servidor sem fragmentação. A ausência de handshake, característica do UDP, é visível pela inexistência de pacotes `SYN/ACK` precedendo os datagramas.

7 Principais Desafios

1. **Diálogo inicial do Cisco IOS:** o *System Configuration Dialog* interceptava os comandos antes do modo privilegiado, exigindo responder **no** antes de iniciar qualquer configuração.
2. **Conflito de sub-redes:** atribuir endereços de interfaces diferentes pertencentes à mesma faixa (/8) gerava erro *overlaps*. A solução foi separar os enlaces com sub-redes distintas.
3. **Sincronização entre threads:** garantir que `TcpServer` e `UdpServer` operassem em paralelo sem condições de corrida nas estruturas de dados compartilhadas do `MonitoringService`.
4. **Definição do protocolo:** criar um protocolo textual simples e extensível, compatível tanto com TCP (orientado a stream) quanto com UDP (orientado a datagrama), exigiu cuidado na delimitação e parsing das mensagens.

8 Conclusão

O projeto *Java Network Monitor* demonstrou na prática os principais conceitos de redes de computadores abordados na disciplina. A aplicação implementa com sucesso a comunicação cliente-servidor com TCP e UDP simultâneos, processamento concorrente com multithreading e interface gráfica com Java Swing.

A análise com Wireshark confirmou o correto funcionamento do protocolo TCP – incluindo handshake, troca de mensagens estruturadas e encerramento de conexão – e a transmissão periódica e eficiente dos datagramas UDP de status. A topologia de três redes distintas, configurada no Cisco Packet Tracer com roteamento estático, validou a comunicação entre redes heterogêneas.

O código-fonte está disponível em:

https://github.com/LINK_DO_GRUPO/java-network-monitor

Uso de Inteligência Artificial

Ferramentas de IA foram utilizadas como apoio em aspectos pontuais: esclarecimento de dúvidas sobre roteamento estático e configuração de roteadores Cisco no Packet Tracer, revisão de trechos de código e auxílio na organização e redação do relatório. A implementação, os testes e a compreensão técnica do projeto foram realizados pelos integrantes do grupo.

Referências

- [1] A. S. Tanenbaum and D. J. Wetherall, *Redes de Computadores*, 5^a ed. Pearson, 2011.
- [2] B. A. Forouzan, *Comunicação de Dados e Redes de Computadores*, 4^a ed. McGraw-Hill, 2008.

- [3] Cisco Systems, *Cisco IOS IP Routing Command Reference*. Disponível em: <https://www.cisco.com>. Acesso em: jun. 2026.
- [4] Oracle Corporation, *Java SE Documentation – java.net package*. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: jun. 2026.
- [5] Grupo Java Network Monitor, *Repositório do Projeto*. Disponível em: https://github.com/LINK_DO_GRUPO/java-network-monitor. Acesso em: jun. 2026.